Lab 4 - Graph Search

Course: CIS 387 - Analysis of Algorithms - Fall 2026
Instructor: Dr. Aaron Mininger

Submission Instructions

Due Date: Nov 15 at 11:59pm

To Submit: Upload a zip file containing all the java code, plus a Word document or pdf with the answers to the analysis questions in parts 1D and 2D. Include your name and the names of anyone you worked with.

Since this is a lab assignment, you are allowed to work with other people. However, you must write and submit your own individual lab report as your personal work. You may consult other resources, including AI, to write the code, but try to work through things yourself first. Since this lab is designed to give you experience with graphs, I encourage you to type up the code yourself by hand.

Setup

All the code and files needed for this homework are available on the course git repository: **https://github.com/amininger-mu/CIS387F26Code**

In the **labs/lab4_graph_search** folder, you will see starter files for each part you should fill in.

---

1. Comparing Graph Representations

In this part, you will be implementing and comparing the three types of graph representations discussed in class:

1. Edge List: A list of **Edge** objects, each connecting two vertices
2. Adjacency Matrix: a 2D array of booleans where **m[i][j]** indicates an edge from i to j
3. Adjacency List: A list of lists, where list *i* contains a list of vertices adjacent to vertex *i*

Open up **Graph.java** to see the Graph interface you will be implementing. It contains 3 methods:

- **int V()**: returns the number of vertices in the graph
- **void addEdge(int v, int w)**: Adds an edge from v to w
- **List<Integer> adj(int v)**: Returns a list of vertices adjacent to vertex *v*

Your task is to implement the 3 classes below. You can use the **DoublingTest** program to test them.

Note: You may assume that these are *simple graphs* (no loops or parallel edges)

A. **`EdgeList.java`**

Implements a graph as a list of edges. (Each **`Edge`** is a **`record Edge(int v1, int v2){ }`**)

The **`EdgeList`** class has 2 instance variables:

- **`ArrayList<Edge> edges`** - the list of edges
- **`int V`** - the number of vertices

Complete the class by implementing the constructor + 3 graph methods.

B. **`AdjacencyMatrix.java`**

Implements a graph using a 2D boolean array (**`m[i][j] == true`** if *i* and *j* are adjacent)

The **`AdjacencyMatrix`** class has 1 instance variable:

- **`boolean[][] m`** - the adjacency matrix

Complete the class by implementing the constructor + 3 graph methods.
Note: if there is an edge between *i* and *j*, both **`m[i][j]`** and **`m[j][i]`** should be true.

C. **`AdjacencyList.java`**

Implements a graph using a list of lists (List *i* contains a list of vertices adjacent to vertex *i*)

The **`AdjacencyList`** class has 1 instance variable:

- **`List<List<Integer>> vertices`**

Complete the class by implementing the constructor + 3 graph methods.
Note: if there is an edge between *i* and *j*, add an entry to both lists **`vertices[i]`** and **`vertices[j]`**.

You should use **`ArrayList`** as the underlying list representation.

D. **`DoublingTest.java`**

When you have finished your implementations, compile run the **`DoublingTest.java`** program. This runs a doubling test on one of the three graph types using the **`MarkConnected`** algorithm on a randomized graph. It prints the time for the 3 stages: creating the graph, adding 10V random edges, and doing the mark connected algorithm (that calls **`graph.adj(v)`** a lot).

Run the program as **`java DoublingTest -m`**

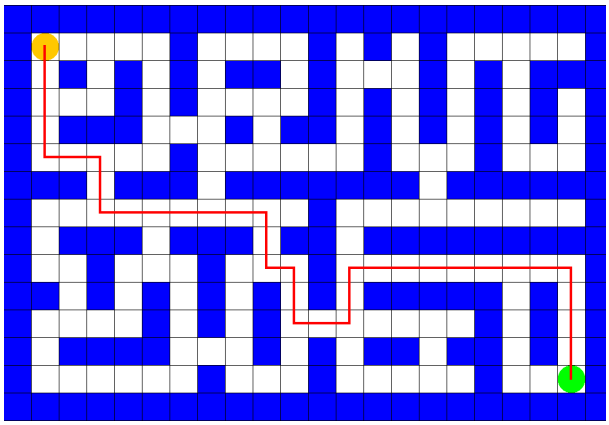
- Use flag **`-m`** (matrix), **`-e`** (edge list), **`-l`** (adj list)

Respond: In your report, paste in the program output from your 3 doubling tests. Then answer the following question:

- *For each graph type, summarize what you observe in the results and explain how/why the specific representation impacts the performance of each stage.*
- *Compare the overall performance between the three types of graphs*

2. Graph Search Algorithms

In this part, you will implement the two graph search algorithms (depth first search and breadth first search) to find a path through a maze.



Project Structure:

File	Description
`PathSearch.java`	The base class for both search algorithms
`BreadthFirstSearch.java`	Implements the BFS algorithm (extends PathSearch)
`DepthFirstSearch.java`	Implements the DFS algorithm (extends PathSearch)
`Maze.java`	A helper class that holds the maze data after loading it from a file
`MazeSolver.java`	The main program that lets you compare searchers on the maze
`mazes/`	A directory containing a few test mazes to solve

Once a maze has been created, the program creates and fills in a **`Graph`** structure, where each vertex is a square in the maze grid, and edges connect two vertices if they are orthogonally adjacent (north, south, east, west) and are both open.

It then uses a graph search algorithm to find a path from the start square to the goal square.

A. `PathSearch.java`

Open up **`PathSearch.java`**, which forms the base class for the two graph search algorithms. This code is based on the book (p. 536 and 540), with some changes.

This class constains three instance variables along with some helper methods:

- **`Graph G`**: a reference to the graph
- **`boolean[] marked`**: where **`marked[i] = true`** if vertex i was visited during the search
- **`int[] edgeTo`**: where **`edgeTo[i] = j`** means **`j`** is the parent vertex along the path (or -1)

Your task is to implement this abstract method in the 2 subclasses:

- **`boolean search(int start, int goal)`**: performs a path search from the start vertex to the goal vertex (updating **`marked`** and **`edgeTo`** arrays). It returns true if a path was found.

B. `BreadthFirstSearch.java`

Implement the **`search`** method in **`BreadFirstSearch.java`**. Base your algorithm on the books (p. 540), but you will have to make a few tweaks:

1. Instead of a **`Queue<Integer>`**, use a **`LinkedList<Integer>`** for the queue.
2. Use linked list methods **`add`** and **`poll`** to model a queue. (instead of **`enqueue`** and **`dequeue`**)
3. Modify the code to return a boolean. (return true immediately when the goal is reached, return false at the end if the search was exhausted)

To test your solution, compile and run the **`MazeSolver.java`** file. You can give it one of the maze files in the **`mazes`** dir for smaller test cases. Then click the **`BFS`** button at the top.

For example: **`java MazeSolver mazes/small_maze.txt`**

C. `DepthFirstSearch.java`

Implement the **`search`** method in **`DepthFirstSearch.java`**. But instead of using recursion like the book, we will use a stack data structure instead.

1. Copy-paste your breadth first search solution.
2. Instead of a queue, rename the **`LinkedList<Integer>`** variable to **`stack`**.
3. Call linked list methods **`push`** and **`pop`** to model a stack.
4. Instead of updating the **`marked`** and **`edgeTo`** arrays when a vertex is pushed, update them when a vertex is popped. (First check that the popped vertex is not already marked, if so, then **`continue`**)

You can test your solution using the **`MazeSolver`** and the **`DFS`** button.

D. Analysis

Run the **`MazeSolver`** without any maze file. The program will generate a random maze. You can also click **`Randomize`** to randomize the maze.

Report: Run both **`DFS`** and **`BFS`** on 5 random mazes, and record the results (the path length and the number of vertices visited during the search). If your algorithms are correct, the results should be significantly different.

Then, summarize the major differences you observe between the two algorithms. Provide explanations for your results (what about the two search algorithms produces the results you see?)